Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

SKILL BASED EVALUATION REPORT

SUBMITTED BY

ANGEL SWEETY P (Reg.No: 5051)
SHALLY S (Reg.No: 5078)
K KOKILA DHARSHINI (Reg.No: 5037)
SELVARANI S (Reg.No: 5088)

25AI2008
MULTILINGUAL AI

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY



OCTOBER 2026

Code-Mixed (Tanglish/Hinglish) Text: Language Identification, Normalization and N-gram/LSTM Language Modelling

Submitted by

ANGEL SWEETY P (Reg.No: 5051)
SHALLY S (Reg.No: 5078)
K KOKILA DHARSHINI (Reg.No: 5037)
SELVARANI S (Reg.No: 5088)



DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be-under Sec-3 of the UGC Act, 1956)
Karunya Nagar, Coimbatore - 641 114. INDIA

OCTOBER 2026

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

BONAFIDE CERTIFICATE

This is to certify that the skill-based evaluation report for the course 25AI2008 Multilingual AI is a bonafide work done during the Odd semester of the academic year 2026-2027 by

Angel Sweety P (Reg.No: 5051)

Shally S (Reg.No: 5078)

K Kokila Dharshini (Reg.No: 5037)

Selvarani S (Reg.No: 5088)

in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (<<AI/AIML>>) of Karunya Institute of Technology and Sciences.

Submitted for the assessment held on _____

Signature of the Class Teacher
<<Class Teacher>> <<Designation>> Division of Artificial Intelligence and Machine Learning

DECLARATION

We hereby declare that the skill-based evaluation report for the course 25AI2008 Multilingual AI submitted by us to Karunya Institute of Technology and Sciences is an authentic work carried out by us under the guidance of <<Course Teacher>>. All the information presented in this report is original and has not been submitted for any other academic purpose.

We acknowledge that the sources of information used in this report have been appropriately cited and referenced. Any contributions from other individuals or organizations have been duly acknowledged.

We understand that any act of plagiarism or academic dishonesty is strictly prohibited and may result in disciplinary action as per the university's regulations.

We take full responsibility for the content and integrity of this report, and we are willing to provide further clarification or information if required.

Signature of student(s)
Angel Sweety P (5051) Shally S (5078) K Kokila Dharshini (5037) Selvarani S (5088)

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

EVALUATION REPORT

S.No	Evaluation Parameter	Max. Marks	Marks Secured
1	Technical Proficiency	10	
2	Problem-solving Ability	5	
3	Creativity and Innovation	5	
4	Communication Skills	5	
5	Teamwork and Presentation	10	
Total		40	

Signature of the Class Teacher
<<Class Teacher>> <<Designation>> Division of Artificial Intelligence and Machine Learning

TABLE OF CONTENTS

S.No		Content Title	Page No
1		Abstract	1
2		Problem Statement	2
3		Methodology/Architecture	4
4	4.1	Implementation	14
	4.2	Coding	20
	4.3	Output Screenshot	24
5		Conclusion	29
6		References	31

ABSTRACT

Social-media users in India routinely write Tamil and Hindi in the Roman alphabet and mix English words into the same sentence. The result, called Tanglish (Tamil-English) and Hinglish (Hindi-English), has no standard spelling, no clear word-level language boundaries and very few annotated resources, so tools built for monolingual text perform poorly on it. This project implements and evaluates three NLP tasks on Romanized Tanglish and Hinglish: token-level language identification, text normalization and next-word language modelling.

Language identification uses two models, a character n-gram TF-IDF classifier with Logistic Regression and a BiLSTM tagger over word embeddings and character-CNN features, plus deterministic rules for numbers, punctuation, URLs, mentions, hashtags and emoji. Normalization is rule- and dictionary-based (Unicode clean-up, repeated-character reduction, slang expansion, a Hinglish dictionary learned from annotator corrections and a curated Tanglish lexicon); it is neither translation nor transliteration. For language modelling, a trigram model with interpolated Kneser-Ney smoothing, written from scratch, is compared with a PyTorch LSTM on the same corpus, vocabulary and evaluation protocol.

Hinglish data come from COMI-LINGUA (CC-BY-4.0; three-annotator labels and references). Tanglish data come from Tanglish-Corpus-185k (CC-BY-4.0), whose token labels are weak (silver) lexicon-based labels, so the Tanglish identifiers are evaluated on a separate 120-sentence hand-reviewed gold set.

Word-level language-identification accuracy is 95.8% (character n-gram) and 96.3% (BiLSTM) for Hinglish, and 95.3% and 95.1% for Tanglish on the hand-reviewed set. Hinglish normalization lowers WER against the best of three references from 13.6% to 10.1%, while the annotators disagree with each other at 16.2% mean WER. The LSTM lowers test perplexity from 193 to 142 (Hinglish) and from 416 to 259 (Tanglish). The limitations are stated explicitly: weak Tanglish labels, a small Tanglish gold set without independent native-speaker validation, and a one-command rebuild that has not been verified end to end.

Keywords: code-mixing, Tanglish, Hinglish, language identification, text normalization, Kneser-Ney, LSTM language model.

PROBLEM STATEMENT

2.1 The code-mixing challenge

Millions of Indian users type Tamil or Hindi with English letters and switch to English in the middle of a sentence, for example *naan inniku college ku late ah varen* (Tanglish) or *kal meeting hai, please notes share kar dena* (Hinglish). Such text is the dominant form of informal communication in comments, chats and posts, yet it breaks the assumptions of standard NLP pipelines, which expect one language, one script and consistent spelling. The main difficulties are:

- **Mixing of English with Tamil and Hindi** inside one sentence, often with Tamil or Hindi grammatical suffixes attached to English words (*college-ku*).
- **Romanized writing instead of native scripts.** Tamil and Hindi are written in Latin letters, so script can no longer separate the languages.
- **Informal spelling variation.** The same word appears as *iruku / irukku / erukku* or *hai / haiii / hain*, and chat slang (*plz, brooo*) is common.
- **Ambiguous words and unclear language boundaries.** Many tokens are valid in more than one language (*than, ah, to, hi*), and names, acronyms and handles belong to no language.
- **Limited gold-standard annotated data.** Public token-level data exist for Hindi-English, but no public gold token-level language-identification corpus was available for Romanized Tamil-English, which forces the use of weak labels.
- **Unseen word combinations.** Because spelling and mixing are so varied, many word sequences at test time never occur in training, which makes language modelling and smoothing difficult.

2.2 Objectives

The project has three measurable objectives, each tied to a metric that is reported in this document:

1. **Token-level language identification.** Label every token of a Tanglish or Hinglish sentence as TA/HI, EN or OTHER (with NUM and PUNCT handled by rules), and

compare a context-free character n-gram model with a context-aware BiLSTM using word-level accuracy and macro-F1 on held-out data.

2. **Consistent code-mixed text normalization.** Map spelling variants to one preferred Romanized spelling without translating, and measure word error rate (WER) against three human references for Hinglish.
3. **Statistical versus neural language modelling.** Compare an interpolated Kneser-Ney trigram model with an LSTM on the same corpus, vocabulary and evaluation positions using perplexity and top-1 / top-5 next-word accuracy.

2.3 Scope

In scope: Romanized Tanglish and Hinglish; the three tasks above; training of all models from scratch on the stated corpora; an interactive Streamlit dashboard that runs locally; honest reporting of evaluation limits.

Out of scope: translation, transliteration into native script, sentiment analysis, speech recognition, transformer or pretrained multilingual-model fine-tuning, and any claim of cross-language transfer between Tanglish and Hinglish models.

METHODOLOGY / ARCHITECTURE

3.1 System overview

The system is organised as four modules that share one preprocessing layer and one inference entry point (Figure 1). Raw text is cleaned and tokenized; each token receives a language label (Module 1); the labels guide the normalizer (Module 2); the normalized words are then scored and extended by two language models (Modules 3 and 4). The same `CodeMixedPipeline` class is used by the Streamlit application, the command-line tool and the evaluation scripts, so the demonstration cannot drift from the reported experiments.

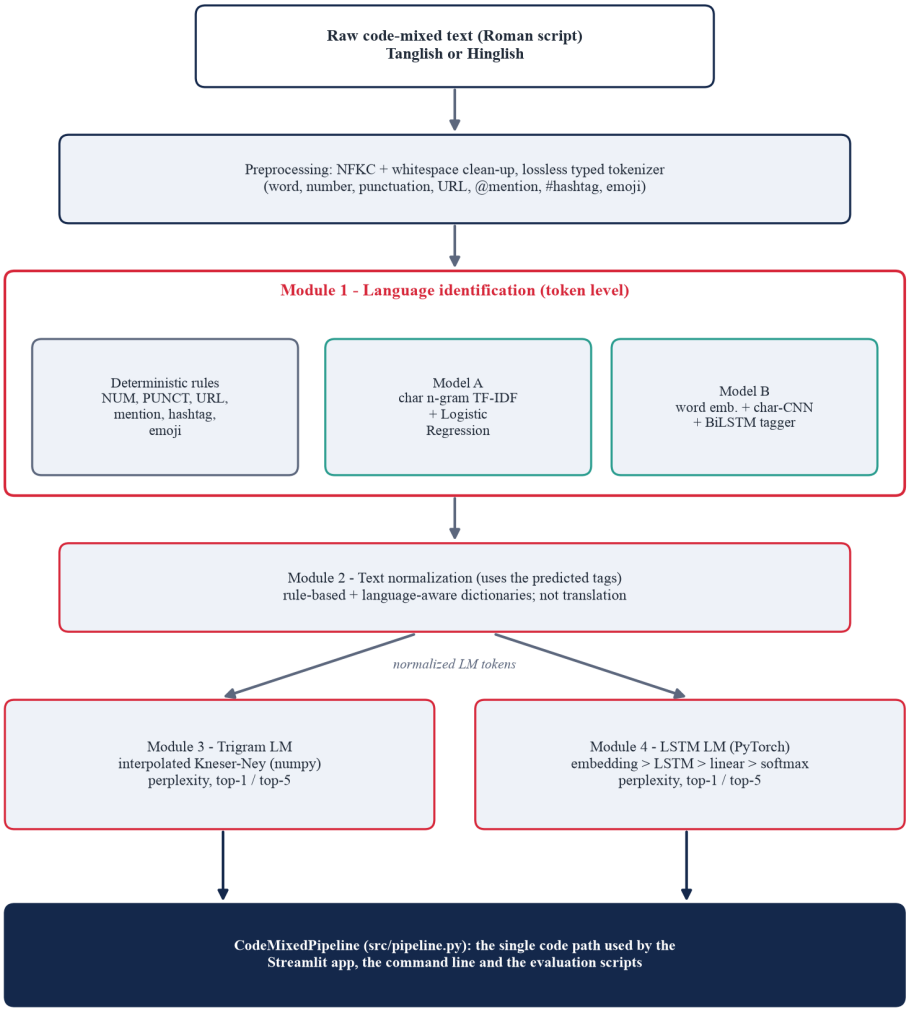


Figure 1. End-to-end architecture of the code-mixed text processing system.

3.2 Datasets and annotation

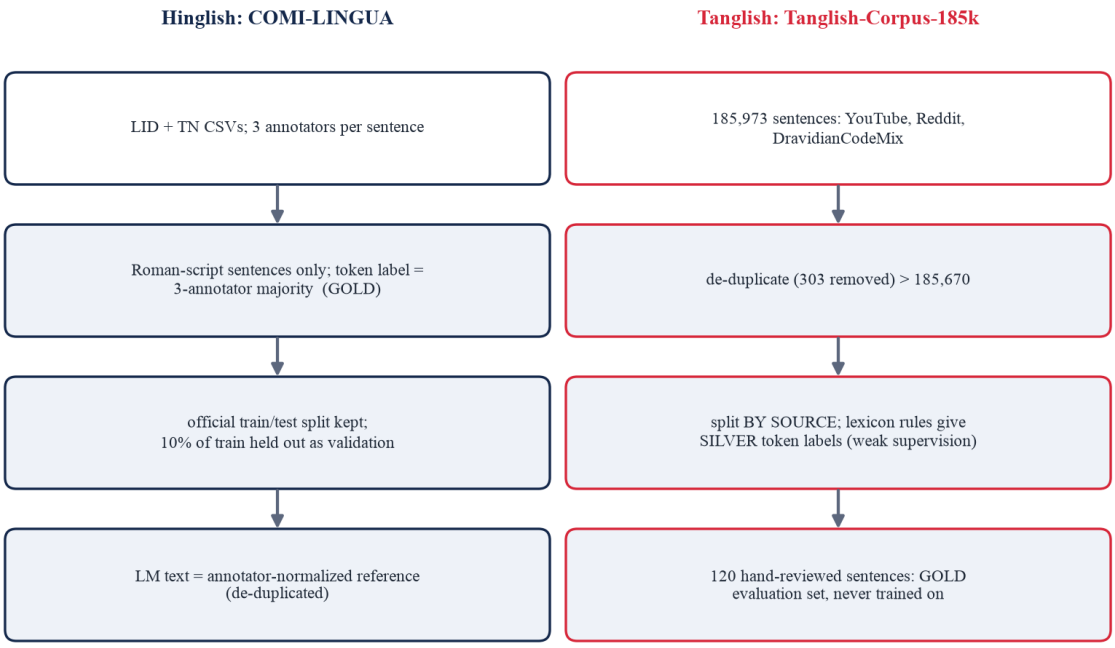


Figure 2. Data sources, splits and label provenance for the two language variants.

Hinglish: COMI-LINGUA

COMI-LINGUA (Sheth et al., Findings of EMNLP 2025) is a manually annotated Hindi-English code-mixed dataset published under CC-BY-4.0 (https://huggingface.co/datasets/LingoIITGN/COMI-LINGUA) [1]. Every instance is annotated by three annotators. The project uses two subsets: *Language Identification (LID)*, where each token carries a Hindi, English or other tag, and *Text Normalization (TN)*, where each sentence has three annotator-written normalized references. Only Roman-script sentences are used, because this project concerns Romanized text: after removing native-script sentences (22,166 training and 4,199 test) and sentences on which the annotators' tokenizations disagree (331 and 51), 7,191 training, 799 validation and 750 test sentences remain for language identification. The token label is the majority of the three annotators (ties fall back to annotator 1). The official train/test split is kept, and validation is a 10% hold-out of train. For language modelling, the annotator-normalized text is used (21,022 / 2,336 / 4,933 sentences after de-duplication).

Tanglish: Tanglish-Corpus-185k

Tanglish-Corpus-185k (https://huggingface.co/datasets/vishnu-n/Tanglish-Corpus-185k, CC-BY-4.0) [12] contains 185,973 Romanized Tamil-English sentences collected from YouTube

comments, Reddit and the DravidianCodeMix corpus; each row carries the text and its source. After lower-cased, punctuation-stripped de-duplication and a minimum-length filter, 185,670 sentences remain (303 removed). The corpus has no token labels, so it is used as follows:

- **Language-model corpus.** The text is normalized by this project's Tanglish normalizer and split *by source* (a whole YouTube video or subreddit goes to a single split, so a comment thread cannot straddle training and test) into 140,682 / 26,297 / 18,622 sentences.
- **Silver token labels for language identification.** 30,000 / 2,000 / 3,000 sentences (train / validation / test) receive weak labels from rules over a curated Tamil lexicon, an English chat-slang lexicon, a named-entity/acronym lexicon and English word frequencies (`wordfreq`), applied in a fixed priority order (Table 1).
- **Gold evaluation set.** 120 sentences (1,327 word tokens) were hand-reviewed and corrected token by token. These sentences are never used for training or tuning, and they are the only honest estimate of Tanglish language-identification accuracy.

Table 1. Priority order of the Tanglish silver-labelling rules (src/silver_labels.py).

Priority	Rule	Label
1	Non-ASCII / mojibake word	OTHER
2	Named-entity / acronym lexicon	OTHER
3	Curated Tamil lexicon	TA
4	English chat-slang lexicon	EN
5	Single letters (a, i, u, r, k are English)	EN / OTHER
6	Common English word (word-frequency zipf >= 3.0)	EN
7	ALL-CAPS token not matched above	OTHER
8	Anything else (out of vocabulary)	TA

Gold versus silver. Silver labels are *not* human annotations: they are the output of the rules above, so a model trained on them can at best reproduce those rules. The 99% score on the silver test split (Table 6) therefore measures agreement with the rules and must not be read as the real accuracy; the hand-reviewed gold figure is the one reported as the Tanglish result. The gold set was reviewed by the project team with an AI assistant and was not validated by independent native-speaker annotators (see Section 5.3).

Label convention

TA Romanized Tamil; **HI** Romanized Hindi; **EN** English, including chat slang and loanwords such as *bro* and *late*; **OTHER** names, acronyms, handles, URLs, emoji and noise (the annotation rule for non-linguistic or other tokens); **NUM** numbers; **PUNCT** punctuation. The classifiers decide only among the word-level labels of the selected variant (TA/EN/OTHER or HI/EN/OTHER); NUM, PUNCT and the non-word kinds of OTHER come from rules.

3.3 Module 1: Language identification

The tokenizer is lossless and typed: every token records whether whitespace preceded it and its kind (WORD, NUM, PUNCT, URL, MENTION, HASHTAG, EMOJI), so a sentence can be rebuilt after editing. Tokens that are not words receive their label from `rule_label` (number, punctuation, URL, @mention, #hashtag and emoji map to NUM, PUNCT or OTHER; a word written in Devanagari or Tamil script maps to HI or TA). Only WORD tokens are passed to a classifier (Figure 3).

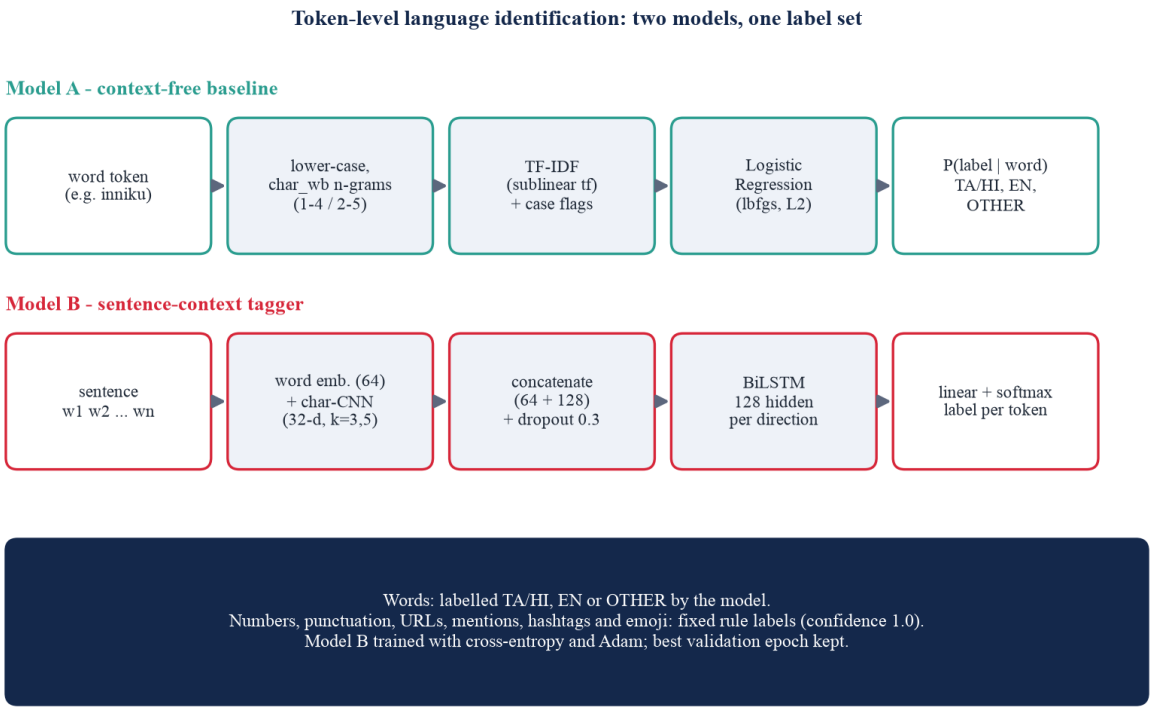


Figure 3. The two language-identification models.

Model A: character n-gram TF-IDF + Logistic Regression

Each word is lower-cased and converted into character n-grams within word boundaries (`char_wb`), weighted by sublinear TF-IDF, and extended with four case features (initial

capital, all capitals, contains a digit, contains an apostrophe). A multinomial Logistic Regression predicts the label. Because the model is context-free, training on unique (word, label) types weighted by their counts is mathematically identical to training on every token occurrence and is much faster. The n-gram range and regularization strength C were chosen on the validation set from a small grid (ranges 1-3, 1-4, 2-5; C in 1, 10, 50): the selected setting was (1,4) with C = 10 for Hinglish and (2,5) with C = 50 for Tanglish.

Model B: word embedding + character CNN + BiLSTM

Model B sees the whole sentence. Each word is represented by a 64-dimensional word embedding (out-of-vocabulary words map to `<unk>`, and 10% of words are randomly replaced by `<unk>` during training so the model relies on spelling as well as identity) concatenated with the output of a character CNN (32-dimensional character embeddings, two convolutions of width 3 and 5 with 64 filters each, max-pooled over characters). The concatenated vector (64 + 128 dimensions) passes through dropout (0.3) into a bidirectional LSTM with 128 hidden units per direction, and a linear layer with softmax assigns a label to every token. Training uses cross-entropy loss, the Adam optimizer, gradient clipping at 5.0 and keeps the epoch with the best validation accuracy.

Important distinction. The BiLSTM above is a *sequence tagger* that predicts a language label for each word. It is a different model, solving a different task, from the LSTM *language model* of Module 4, which predicts the next word.

3.4 Module 2: Text normalization

Normalization rewrites noisy spellings into one preferred Romanized spelling so that the same word is represented consistently. It is **not translation** (Tamil and Hindi words are never replaced by English) and **not transliteration** (nothing is converted to Tamil or Devanagari script). The pipeline (Figure 4) applies the following steps in order:

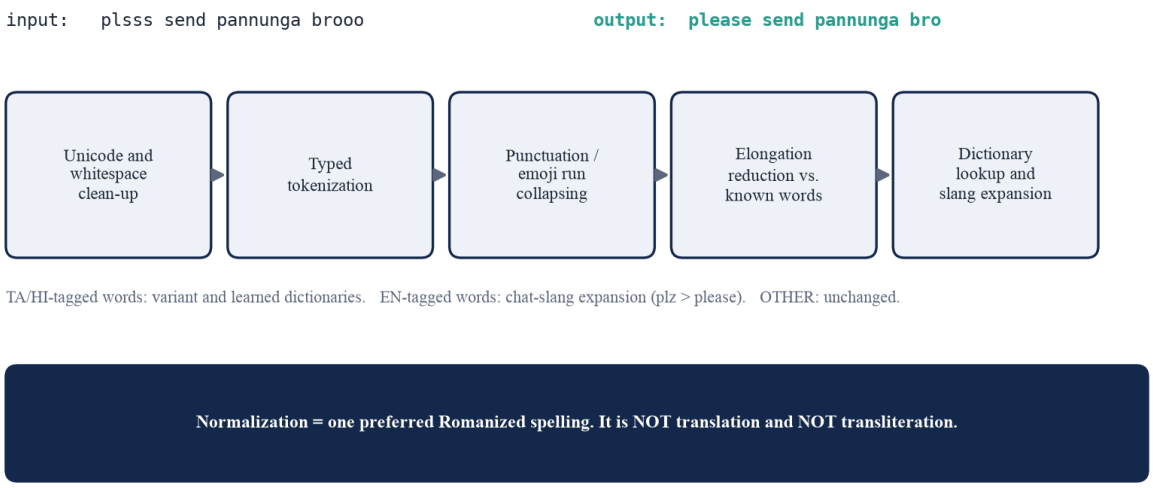


Figure 4. The rule- and dictionary-based normalization pipeline.

- **Unicode and whitespace handling:** NFKC normalization, removal of zero-width characters, unification of quotation marks and collapsing of whitespace.
- **Tokenization and detokenization:** the typed tokenizer splits text; `detokenize` rebuilds it with correct spacing around punctuation.
- **Punctuation normalization:** runs of the same punctuation or emoji are collapsed (`!!!` becomes `!`, `....` becomes `...`).
- **Repeated-character reduction:** only runs of three or more identical letters are touched, and each run may shrink to one or two characters; the candidate that is a known word is chosen (`rombbba` to `romba`, `haiii` to `hai`), so legitimate double letters (`irukku`, `good`) are never altered.
- **English chat-slang expansion:** applied to English-tagged or untagged words (`plz` to `please`).
- **Language-aware spelling dictionaries:** for Tanglish, a curated lexicon of variant groups maps spellings such as `iruku` and `erukku` to `irukku`; for Hinglish, a dictionary is learned automatically from the COMI-LINGUA annotator corrections by aligning each original sentence with its references and keeping rewrites that are frequent enough (minimum count 2 and minimum rewrite ratio 0.4, chosen on validation data; 1,683 entries). Dictionary entries respect the predicted language tag, so a Tamil-tagged word is not rewritten by an English rule.

Words tagged OTHER (names, handles, acronyms) are left unchanged. The Hinglish dictionary is derived only from the training split's annotator references; the Tanglish lexicon is hand-curated from the most frequent tokens of Tanglish-Corpus-185k.

3.5 Module 3: Trigram language model

A language model assigns a probability to a word sequence. Under the trigram assumption, the probability of a word *w* depends only on the two previous words *u v*, P(w | u v), and is estimated from n-gram frequency counts in the training corpus: unigram counts c(w), bigram counts c(v w) and trigram counts c(u v w). Raw relative frequencies assign zero probability to unseen trigrams, which are very common in noisy code-mixed text, so smoothing is essential. The model implemented here is **interpolated Kneser-Ney (KN)** [7], written from scratch with sorted NumPy arrays (no NLTK or external language-model library). The highest order uses discounted counts, and the probability mass removed by the absolute discount D is redistributed to a lower-order model:

$$P_{KN}(w_i \mid w_{i-2}w_{i-1}) = \frac{\max\left(c(w_{i-2}w_{i-1}w_i) - D,\ 0\right)}{c(w_{i-2}w_{i-1}\bullet)} + \gamma(w_{i-2}w_{i-1})\, P_{KN}(w_i \mid w_{i-1}) \tag{1}$$

$$\gamma(uv) = \frac{D \cdot N_{1+}(uv\bullet)}{c(uv\bullet)} \tag{1a}$$

Lower orders use *continuation counts* instead of raw counts. N1+(• v w) is the number of distinct words that precede the bigram *v w*, which measures how freely a word can start new contexts (the classic example is that *Francisco* is frequent but follows almost only *San*):

$$P_{KN}(w \mid v) = \frac{\max\left(N_{1+}(\bullet vw) - D,\ 0\right)}{N_{1+}(\bullet v\bullet)} + \frac{D \cdot N_{1+}(v\bullet)}{N_{1+}(\bullet v\bullet)}\, P_{KN}(w) \tag{2}$$

$$P_{KN}(w) = \frac{\max\left(N_{1+}(\bullet w) - D,\ 0\right)}{N_{1+}(\bullet\bullet)} + \frac{D \cdot |\{w : N_{1+}(\bullet w) > 0\}|}{N_{1+}(\bullet\bullet)} \cdot \frac{1}{|V|} \tag{3}$$

The final uniform term 1/|V| guarantees that every vocabulary word, including `<unk>`, has non-zero probability. Unseen contexts back off to the next lower order. The discount D is tuned on validation perplexity over the grid {0.5, 0.6, 0.7, 0.75, 0.8, 0.9}; the best value was 0.9 for both variants, which is the upper edge of the grid, so a larger value was not tried. **Prediction:** the top-1 and top-5 next words are exact: the candidates are the trigram followers, the bigram followers and the most probable unigrams, which provably contain the

top-k under this back-off scheme. **Evaluation:** perplexity over all test tokens, including `</s>` and `<unk>`:

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(w_i \mid w_{i-2}, w_{i-1})\right) \tag{4}$$

3.6 Module 4: LSTM language model

The neural language model reads the same word sequence and predicts the next word with a recurrent network (Figure 5). Its components are:

- **Token vocabulary.** The vocabulary is built once from the training split (words with at least `min_count` occurrences, capped at a maximum size) plus the special tokens `<pad>`, `<unk>`, `<s>` and `</s>`; numbers are mapped to `<num>`.
- **Embedding layer and dropout.** Word indices are mapped to dense vectors (256 dimensions for Tanglish, 200 for Hinglish) and regularized with dropout (0.3 and 0.4).
- **LSTM layer** [8]: one layer with 512 (Tanglish) or 400 (Hinglish) hidden units.
- **Linear projection and softmax.** The hidden state is projected back to the embedding size and multiplied by the transposed embedding matrix (weight tying [10]), then a softmax gives the probability of every vocabulary word.
- **Training.** Cross-entropy loss with teacher forcing, the Adam optimizer [11] (learning rate 0.002), backpropagation through time over each padded sentence batch, gradient-norm clipping at 1.0, and a learning-rate reduction on plateau.
- **Validation and checkpoint selection.** Validation perplexity is computed after every epoch; the weights from the epoch with the lowest validation perplexity are restored and saved, and training stops early if validation perplexity stops improving.

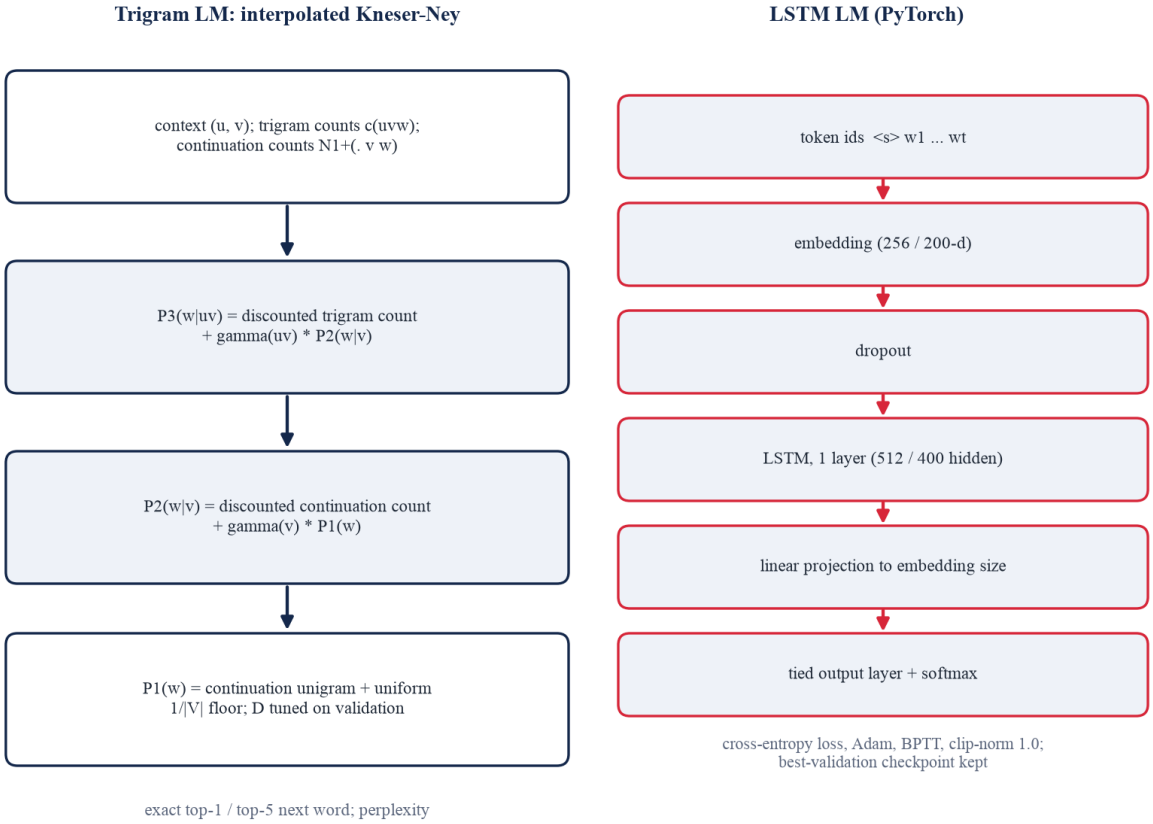


Figure 5. Architecture of the Kneser-Ney trigram model (left) and the LSTM language model (right).

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \tag{5}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t) \tag{5a}$$

$$P(w_{t+1} \mid w_{\le t}) = \mathrm{softmax}\left(E^{\top} W_p \,\mathrm{Drop}(h_t) + b\right), \qquad \mathcal{L} = -\tfrac{1}{N}\sum_t \log P(w_{t+1} \mid w_{\le t}) \tag{6}$$

Fair comparison. Within each language variant the trigram model and the LSTM use the *same* training corpus, the *same* vocabulary object (built once from the training split), the same `<s>` … `</s>` sentence framing, the same test sentences, and the same scoring: perplexity over every token, and top-1/top-5 accuracy over positions whose true next word is a real vocabulary word (`<unk>`, `<num>` and `</s>` targets are skipped). The training script asserts that both models are scored on an identical number of positions (101,053 for Hinglish and 268,856 for Tanglish).

3.7 Evaluation protocol and metrics

- **Language identification:** word-level accuracy and macro-F1 on WORD tokens (also reported over all tokens), with per-class precision/recall and confusion matrices; Hinglish on the gold COMI-LINGUA test split; Tanglish on the hand-reviewed gold set (primary) and on the silver test split (reported separately and flagged as inflated).
- **Normalization:** corpus WER (and CER) on lower-cased words without punctuation (WER = (S + D + I) / N, where S, D and I count substituted, deleted and inserted words and N is the number of reference words), against the first reference and against the best of the three references; exact-match rate; inter-annotator WER as a ceiling reference.
- **Language modelling:** perplexity, top-1 and top-5 next-word accuracy.

3.8 Individual contribution of team members

The template asks each team to highlight individual contributions. The repository does not record who wrote which module, so this table is intentionally left for the team to complete; no contributions have been assumed.

Team member	Reg.No	Contribution (to be filled by the team)
Angel Sweety P	5051	<<to be filled>>
Shally S	5078	<<to be filled>>
K Kokila Dharshini	5037	<<to be filled>>
Selvarani S	5088	<<to be filled>>

IMPLEMENTATION – CODING AND OUTPUT SCREENSHOT

4.1 Implementation

Software stack

The software environment was read from `requirements.txt` and from the virtual environment of the project (Table 2). The n-gram model, the evaluation metrics (WER/CER, perplexity, top-k) and the tokenizer are written from scratch with NumPy; NLTK is not used. Matplotlib is listed in `requirements.txt` as optional (used for the exploration notebook only).

Table 2. Software stack.

Component	Version	Used for
Python	3.14.3	implementation language
PyTorch	2.14.0	BiLSTM tagger, LSTM language model (MPS backend for training)
scikit-learn	1.9.1	TF-IDF, Logistic Regression, metrics
NumPy / pandas	2.4.0 / 3.0.3	KN trigram model, data handling
wordfreq	(requirements: >= 3.1)	English word frequencies for silver labels and the normalizer
joblib	(requirements: >= 1.4)	saving scikit-learn models
Streamlit	1.62.0	interactive dashboard (app.py)
pytest	>= 8	25 unit and integration tests (run: 25 passed)

Project structure

Path	Contents
app.py	Streamlit dashboard: Analyze, Batch, Evaluation and About pages
src/preprocessing.py	typed lossless tokenizer, rule labels, elongation candidates
src/language_identifier.py, train_lid.py	Model A (char n-gram + LogReg) and Model B (BiLSTM tagger); training
src/normalizer.py, tanglish_lexicon.py, train_normalizer.py	rule/dictionary normalizer, curated Tanglish lexicon, dictionary learning
src/ngram_model.py	interpolated Kneser-Ney trigram model (NumPy)
src/lstm_model.py, train_lm.py, vocab.py	LSTM language model, shared vocabulary, joint LM training and evaluation
src/evaluation.py, pipeline.py, run_all.py	metrics; single inference entry point; stage runner
data/	raw/ (downloaded), processed/ (splits), lexicons/, gold/ (Tanglish gold files)
models/, results/, tests/	trained models; metric JSON files and screenshots; 25 tests

Hardware and training configuration

Training and evaluation were run on an Apple M4 laptop with 16 GB of memory under macOS (hardware read from the development machine); the neural models were trained on PyTorch's MPS (Apple GPU) backend, as recorded in the saved results. The BiLSTM taggers were trained for 10 epochs (batch 64, Adam, learning rate 0.002). The LSTM language models used the settings in Table 3. The Tanglish LSTM vocabulary is capped at 20,000 words (10.9% of test tokens are out of vocabulary) because a 30,000-word softmax froze training on the MPS backend; the cap is applied identically to the trigram model. The Tanglish LSTM trained for 10 epochs (validation perplexity was still falling slowly), the Hinglish LSTM stopped early after 25 epochs.

Table 3. Language-model configuration and size (from results/lm_*.json).

	Hinglish	Tanglish
Train / val / test sentences	21,022 / 2,336 / 4,933	140,682 / 26,297 / 18,622
Vocabulary (min count)	10,140 (2)	20,000 (3, capped at 20,000)
Test out-of-vocabulary rate	3.9%	10.9%
Trigram types / discount D	284,022 / 0.9	1,813,996 / 0.9
LSTM embedding / hidden / dropout	200 / 400 / 0.4	256 / 512 / 0.3
LSTM parameters	3,081,540	6,848,288
LSTM epochs run / training time	25 (early stop) / 233 s	10 / 1206 s

Experimental results

All numbers below were read from the saved files `results/lid_*.json`, `results/normalization.json` and `results/lm_*.json`. They describe these datasets and splits and are not general accuracy claims.

Table 4. Language identification: word-level accuracy and macro-F1 on held-out data.

Model	Hinglish gold test acc. / macro-F1	Tanglish hand-reviewed gold (120 sent.) acc. / macro-F1	Tanglish silver test (inflated) acc. / macro-F1
Character n-gram + TF-IDF + LogReg	95.8% / 0.889	95.3% / 0.857	99.0% / 0.989
BiLSTM (word + char-CNN)	96.3% / 0.906	95.1% / 0.850	98.8% / 0.987

The Tanglish gold figures come from a hand-reviewed set, not from an independently annotated public benchmark. The silver-test figures (about 99%) measure agreement with the labelling rules and are shown only to make that inflation visible. Per-class results show where errors concentrate: on the Tanglish gold set the OTHER class (names and acronyms) has recall of only 47.5% (character n-gram) and 45.0% (BiLSTM) on 80 tokens, i.e. roughly 46%, whereas TA and EN recall are above 97% and 99%; on Hinglish the OTHER recall is 66.3% and 73.0%.

Table 5. Hinglish normalization on the COMI-LINGUA TN test split (4,934 sentences).

System	WER vs best of 3 references	WER vs first reference	Exact match (best reference)
No normalization (baseline)	13.6%	24.1%	18.2%
Rules only (punctuation + elongation)	13.2%	23.6%	19.1%
Rules + learned dictionary (final)	10.1%	18.2%	21.6%
Mean inter-annotator WER (reference ceiling)	16.2%		

Normalization WER falls from 13.6% to 10.1%. The three human annotators disagree with one another at a mean WER of 16.2%, which shows that no single correct spelling exists; normalization evaluation is therefore partly subjective, and scores should be read against this disagreement level and not against zero.

Table 6. Language modelling on the held-out test sentences (same corpus, vocabulary and positions for both models).

Metric	Hinglish Trigram KN	Hinglish LSTM	Tanglish Trigram KN	Tanglish LSTM
Test perplexity (lower is better)	193	142	416	259
Top-1 next-word accuracy	17.8%	19.0%	11.5%	12.8%
Top-5 next-word accuracy	34.9%	37.0%	23.2%	25.4%

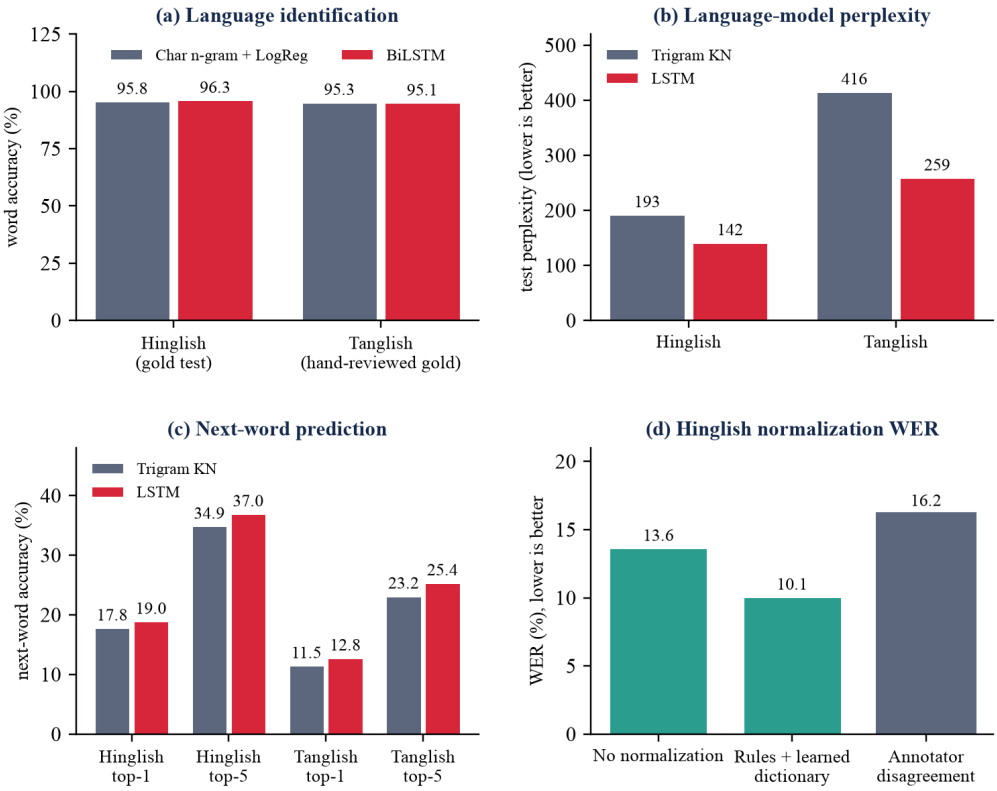


Figure 6. Summary of results: (a) language identification, (b) perplexity, (c) next-word accuracy, (d) Hinglish normalization WER. Tanglish language identification uses the hand-reviewed gold set.

The LSTM reduces perplexity by 26% for Hinglish and 38% for Tanglish, but improves next-word accuracy by only about one to two points: the distributions are better calibrated, while the single most likely word is often the same. Perplexity does not mean that generated sentences are meaningful; the corpora are noisy social-media text. Tanglish normalization was checked against 68 hand-written reference pairs and reached 97.1% exact match (WER 1.4%), but those references were written with the same dictionary conventions as the system, so this is a consistency check and **not** independent evidence of generalization. Cross-variant checks show that the models do not transfer reliably: the Hinglish identifier reaches 96.3% on Hinglish but 63.1% on Tanglish text, and the Hinglish model scores 85.6% on the Tanglish gold set (English versus native-language words only).

Testing and reproducibility status

The 25 unit and integration tests in `tests/` pass (`python -m pytest tests -q`). The stages of the pipeline (data preparation, language identification, normalization, language models) were each run and their results saved separately. The one-command rebuild `python -m src.run_all` has **not** been verified end to end in a single clean run, and the training

pipeline was modified after some stages had been run; the reported numbers come from the saved result files, not from a fresh full rebuild. No model was retrained while preparing this report.

4.2 Coding

Short, representative excerpts of the actual source files are shown below (the complete code is not reproduced). Each excerpt is copied unchanged from the repository.

Listing 1. Typed tokenizer and deterministic labels for non-word tokens (src/preprocessing.py).

```
_TOKEN_RE = re.compile(
    rf"""
    (?P<URL>(?:https?://|www\.)\S+)
  | (?P<MENTION>@\w+)
  | (?P<HASHTAG>\#\w+)
  | (?P<NUM>\d+(?:[.,:/]\d+)*(?!\w))
  | (?P<WORD>\w+(?:['’]\w+)*)
  | (?P<EMOJI>[{_EMOJI_RANGES}]+)
  | (?P<PUNCT>[^\w\s])
    """,
    re.VERBOSE | re.UNICODE,
)

def rule_label(tok: Token) -> str | None:
    """Deterministic label for non-word tokens; ``None`` for words."""
    if tok.kind == NUM:
        return "NUM"
    if tok.kind == PUNCT:
        return "PUNCT"
    if tok.kind in (URL, MENTION, HASHTAG, EMOJI):
        return "OTHER"
    if tok.kind == WORD and not tok.text.isascii():
        if _DEVANAGARI.search(tok.text):
            return "HI"
        if _TAMIL.search(tok.text):
            return "TA"
        return "OTHER"          # accented letters / mojibake noise
    return None
```

Listing 2. Model A: character n-gram TF-IDF + Logistic Regression (src/language_identifier.py).

```
class CharNgramLID:
    def _x(self, words: list[str], fit: bool = False):
        low = [w.lower() for w in words]
        if fit:
            self.vec = TfidfVectorizer(analyzer="char_wb", ngram_range=self.ngram_range,
                                       sublinear_tf=True, min_df=1, dtype=np.float64)
            X = self.vec.fit_transform(low)
        else:
            X = self.vec.transform(low)
        return sp.hstack([X, _case_feats(words)], format="csr")

    def fit(self, sentences: list[list[str]], labels: list[list[str]]) -> "CharNgramLID":
        # The model is context-free, so training on unique (token, label) types
        # weighted by their counts is mathematically identical to training on
        # every token occurrence, and much faster.
        cnt = Counter((w, l) for s, ls in zip(sentences, labels) for w, l in zip(s, ls))
        items = list(cnt.items())
        words = [w for (w, _), _ in items]
        y = [l for (_, l), _ in items]
        wts = np.asarray([c for _, c in items], dtype=np.float64)
        self.clf = LogisticRegression(C=self.C, max_iter=400, solver="lbfgs")
        self.clf.fit(self._x(words, fit=True), y, sample_weight=wts)
        return self
```

Listing 3. Model B: word embedding + character CNN + BiLSTM tagger (src/language_identifier.py).

```
class _Tagger(nn.Module):
    def __init__(self, n_words, n_chars, n_labels, w_dim=64, c_dim=32, c_filters=64,
                 hidden=128, dropout=0.3):
        super().__init__()
        self.cfg = dict(n_words=n_words, n_chars=n_chars, n_labels=n_labels, w_dim=w_dim,
                        c_dim=c_dim, c_filters=c_filters, hidden=hidden, dropout=dropout)
        self.wemb = nn.Embedding(n_words, w_dim, padding_idx=0)
        self.cemb = nn.Embedding(n_chars, c_dim, padding_idx=0)
        self.conv3 = nn.Conv1d(c_dim, c_filters, 3, padding=1)
        self.conv5 = nn.Conv1d(c_dim, c_filters, 5, padding=2)
        self.drop = nn.Dropout(dropout)
        self.lstm = nn.LSTM(w_dim + 2 * c_filters, hidden, batch_first=True, bidirectional=True)
        self.out = nn.Linear(2 * hidden, n_labels)

    def forward(self, words, chars):
        B, T, L = chars.shape
        c = self.cemb(chars.view(B * T, L)).transpose(1, 2)              # (BT, c_dim, L)
```

```
        mask = (chars.view(B * T, 1, L) != 0)
        c3 = F.relu(self.conv3(c)).masked_fill(~mask, -1e4).max(-1).values
        c5 = F.relu(self.conv5(c)).masked_fill(~mask, -1e4).max(-1).values
        c = torch.cat([c3, c5], -1).view(B, T, -1)
        c = torch.where(c < -1e3, torch.zeros_like(c), c)                # padded tokens
        h = self.drop(torch.cat([self.wemb(words), c], -1))
        h, _ = self.lstm(h)
        return self.out(self.drop(h))
```

Listing 4. Merging rule labels with model predictions (src/language_identifier.py).

```
def identify_tokens(model, tokens: list[Token]) -> list[tuple[str, float]]:
    """Label every token: rules for NUM/PUNCT/URL/..., ``model`` for words.

    Returns ``(label, confidence)``; rule-based labels have confidence 1.0.
    """
    word_idx = [i for i, t in enumerate(tokens) if rule_label(t) is None]
    preds = model.predict_sentence([tokens[i].text for i in word_idx]) if word_idx else []
    out: list[tuple[str, float]] = [("OTHER", 1.0)] * len(tokens)
    for i, t in enumerate(tokens):
        rl = rule_label(t)
        if rl:
            out[i] = (rl, 1.0)
    for i, p in zip(word_idx, preds):
        out[i] = p
    return out
```

Listing 5. Weak (silver) labelling of Tanglish words (src/silver_labels.py).

```
def _lookup(w: str, original: str) -> str | None:
    if w in NAMED_ENTITIES:
        return "OTHER"
    if w in TAMIL_WORDS:
        return "TA"
    if w in EN_SLANG:
        return "EN"
    if len(w) == 1:
        return "EN" if w in _SINGLE_EN else "OTHER"
    if _is_english(w):
        return "EN"
    return None

def silver_label_word(text: str) -> str:
    """Label one WORD token (any casing) as TA / EN / OTHER."""
    if _NON_ASCII.search(text):
        return "OTHER"
    w = text.lower()
    lab = _lookup(w, text)
    if lab:
        return lab
    for cand in elongation_candidates(w):
        lab = _lookup(cand, text)
        if lab:
            return lab
    if len(text) >= 2 and text.isupper():
        return "OTHER"
    return "TA"
```

Listing 6. Language-aware normalization of one word (src/normalizer.py).

```
def _reduce_elongation(self, w: str) -> tuple[str, bool]:
    cands = elongation_candidates(w)
    if not cands:
        return w, False
    scored = [(self.known.get(c, 0) + (1 if self._is_known(c) else 0), -len(c), c) for c in cands]
    best = max(scored)
    if best[0] > 0:
        return best[2], True
    return max(cands, key=len), True      # conservative: keep a double letter

def _lookup(self, w: str, tag: str | None) -> tuple[str, str] | None:
    if tag != "EN":
        tgt = self.variants.get(w)
        # variants that are also English words (e.g. "than") need a TA/HI tag
        if tgt and (tag is not None or not _is_english(w)):
            return tgt, "variant"
    if tag in ("EN", None):
        tgt = self.slang.get(w)
        if tgt:
            return tgt, "slang"
    ent = self.learned.get(w)
    if ent:
        tgt, lang = ent
        if tag is None or lang is None or lang == tag:
            return tgt, "learned"
    return None

def normalize_word(self, text: str, tag: str | None = None) -> tuple[str, str | None]:
```

```
    if tag == "OTHER":
        return text, None
    lw = text.lower()
    rule, cur = None, lw
    cur, did = self._reduce_elongation(lw)
    rule = "elongation" if did and cur != lw else None
    hit = self._lookup(cur, tag)
    if hit:
        cur, rule = hit[0], hit[1]
    if cur == lw:
        return text, None
    return _match_case(text, cur), rule
```

Listing 7. Interpolated Kneser-Ney trigram probability, vectorised (src/ngram_model.py).

```
def _refresh_p1(self) -> None:
    D = self.D
    self.p1 = (np.maximum(self._cont - D, 0) / self._cont_total
               + D * self._cont_types / self._cont_total / self.V)
    # <pad>/<s> are never predicted; keep the distribution proper over the rest
    self.p1 = self.p1 / self.p1.sum()

def prob_batch(self, u: np.ndarray, v: np.ndarray, w: np.ndarray) -> np.ndarray:
    """P(w | u, v) for aligned arrays (vectorised)."""
    D, V = self.D, self.V
    u, v, w = (np.asarray(x, dtype=np.int64) for x in (u, v, w))
    p1 = self.p1[w]
    # bigram level
    tot2 = _lookup(self.ctx2, self.ctx2_total, v)
    typ2 = _lookup(self.ctx2, self.ctx2_types, v)
    c2 = _lookup(self.k2, self.c2, v * V + w).astype(np.float64)
    safe2 = np.where(tot2 > 0, tot2, 1.0)
    p2 = np.where(tot2 > 0, np.maximum(c2 - D, 0) / safe2 + D * typ2 / safe2 * p1, p1)
    # trigram level
    uv = u * V + v
    tot3 = _lookup(self.ctx3, self.ctx3_total, uv)
    typ3 = _lookup(self.ctx3, self.ctx3_types, uv)
    c3 = _lookup(self.k3, self.c3, uv * V + w).astype(np.float64)
    safe3 = np.where(tot3 > 0, tot3, 1.0)
    return np.where(tot3 > 0, np.maximum(c3 - D, 0) / safe3 + D * typ3 / safe3 * p2, p2)

def tune_discount(self, val: list[list[int]], grid=(0.5, 0.6, 0.7, 0.75, 0.8, 0.9)) -> dict:
    scores = {}
    for d in grid:
        self.set_discount(d)
        scores[d] = self.perplexity(val)
    best = min(scores, key=scores.get)
    self.set_discount(best)
    return scores
```

Listing 8. LSTM language model definition and next-word prediction (src/lstm_model.py).

```
class LSTMLM(nn.Module):
    def __init__(self, vocab_size: int, emb: int = 256, hidden: int = 512,
                 layers: int = 1, dropout: float = 0.3, pad_id: int = 0):
        super().__init__()
        self.cfg = dict(vocab_size=vocab_size, emb=emb, hidden=hidden,
                        layers=layers, dropout=dropout, pad_id=pad_id)
        self.embedding = nn.Embedding(vocab_size, emb, padding_idx=pad_id)
        self.drop = nn.Dropout(dropout)
        self.lstm = nn.LSTM(emb, hidden, num_layers=layers, batch_first=True,
                            dropout=dropout if layers > 1 else 0.0)
        self.proj = nn.Linear(hidden, emb)
        self.out = nn.Linear(emb, vocab_size)
        self.out.weight = self.embedding.weight   # weight tying
        nn.init.uniform_(self.embedding.weight, -0.1, 0.1)

    def forward(self, x: torch.Tensor, state=None):
        h = self.drop(self.embedding(x))
        h, state = self.lstm(h, state)
        h = self.drop(self.proj(self.drop(h)))
        return self.out(h), state

@torch.no_grad()
def predict_next(self, context_ids: list[int], k: int = 5) -> list[tuple[str, float]]:
    x = torch.tensor([[self.vocab.bos] + list(context_ids)], device=self.device)
    logits, _ = self.model(x)
    logp = F.log_softmax(logits[0, -1], dim=-1)          # full-vocabulary probabilities
    top = torch.topk(logp + self._mask, k)
    return [(self.vocab.itos[i], math.exp(logp[i].item())) for i in top.indices.tolist()]
```

Listing 9. Core of the LSTM training step with checkpoint selection (src/lstm_model.py, excerpt of train_lstm).

```
        for bi, (_, batch) in enumerate(_batches(tr, max_tokens, vocab.pad, True, rng)):
            batch = batch.to(device)
            logits, _ = model(batch[:, :-1])
            tgt = batch[:, 1:]
```

```
            loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)), tgt.reshape(-1),
                                   ignore_index=vocab.pad)
            opt.zero_grad(set_to_none=True)
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), clip)
            opt.step()
            n = (tgt != vocab.pad).sum().item()
            tot += loss.item() * n; cnt += n
        ...
        val_nll, _ = evaluate_nll(model, va, vocab.pad, device)
        sched.step(val_nll)
        rec = dict(epoch=ep, train_ppl=math.exp(tot / cnt), val_ppl=math.exp(val_nll),
                   lr=opt.param_groups[0]["lr"], seconds=time.time() - te)
        history.append(rec)
        log(f"  epoch {ep}: train ppl {rec['train_ppl']:.1f}  val ppl {rec['val_ppl']:.1f}  "
            f"({rec['seconds']:.0f}s)")
        if val_nll < best - 1e-4:
            best, bad = val_nll, 0
            best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
        else:
            bad += 1
    ...
    model.load_state_dict(best_state)
```

Listing 10. Word error rate and the top-k next-word protocol shared by both language models (src/evaluation.py).

```
def wer_cer(hyp: str, ref: str) -> tuple[int, int, int, int]:
    """(word_errors, ref_words, char_errors, ref_chars)."""
    h, r = _canon(hyp), _canon(ref)
    hs, rs = " ".join(h), " ".join(r)
    return edit_distance(h, r), len(r), edit_distance(hs, rs), len(rs)

def topk_accuracy_ngram(lm, sentences: list[list[int]], ks=(1, 5), skip: set[int] = frozenset()):
    """Same protocol as ``LSTMPredictor.topk_accuracy``: next-word accuracy over positions
    whose target is a real word (``<unk>``, ``<num>`` and ``</s>`` targets are ignored)."""
    kmax = max(ks)
    hits = {k: 0 for k in ks}
    total = 0
    for s in sentences:
        ctx: list[int] = []
        for w in s:
            if w not in skip:
                top = [lm.vocab.stoi[x] for x, _ in lm.predict_next(ctx, k=kmax)]
                for k in ks:
                    hits[k] += w in top[:k]
                total += 1
            ctx.append(w)
    return {k: hits[k] / total for k in ks}, total
```

Listing 11. Single inference entry point used by the app, CLI and evaluation (src/pipeline.py, excerpt).

```
def analyze(self, text, lid_model="char_ngram", k=5):
    ...
    toks, labels, conf = self.identify(text, lid_model)
    if not toks:
        raise ValueError("No tokens found in the input.")
    norm = self.normalizer.normalize(text, labels)
    ctx = lm_tokens(norm.text)
    ids = self.vocab.encode(ctx) if self.vocab else []
    oov = [w for w, i in zip(ctx, ids) if self.vocab and i == self.vocab.unk]
    res = Analysis(
        text=text, variant=self.variant, tokens=[t.text for t in toks], labels=labels,
        confidences=conf, normalized=norm,
        code_mixing_index=code_mixing_index(labels), summary=_describe(self.variant, labels),
        lm_context=ctx, oov_words=oov)
    if self.vocab and ids:
        ids = ids[-50:]          # long inputs: only the recent context matters
        if self.ngram:
            res.ngram = self.ngram.predict_next(ids, k)
        if self.lstm:
            res.lstm = self.lstm.predict_next(ids, k)
    ...  # fluency scoring and return
```

4.3 Output Screenshot

All figures below are genuine screenshots of the project's Streamlit dashboard, cropped only to remove empty margins and the navigation sidebar. Figures 7, 9, 10 and 12 are the files already stored in the project's `results/` folder; Figures 8, 11 and 13 were captured from the same running application (inference only, no retraining). No screenshot is generated or edited.

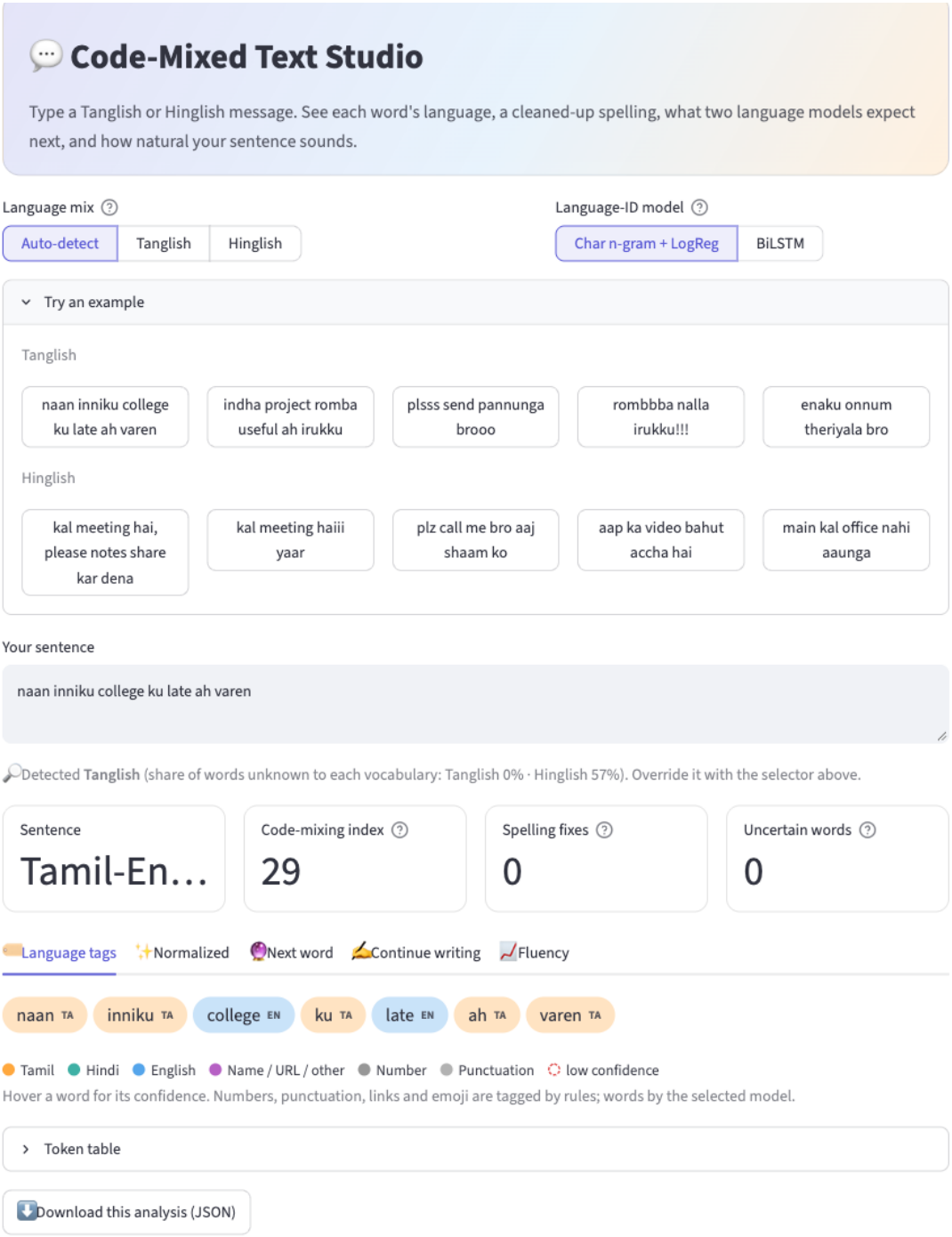


Figure 7. Main dashboard (Analyze page): example inputs, Tanglish sentence, auto-detected variant, code-mixing index and token-level language identification (naan[TA] inniku[TA] college[EN] ku[TA] late[EN] ah[TA] varen[TA]).

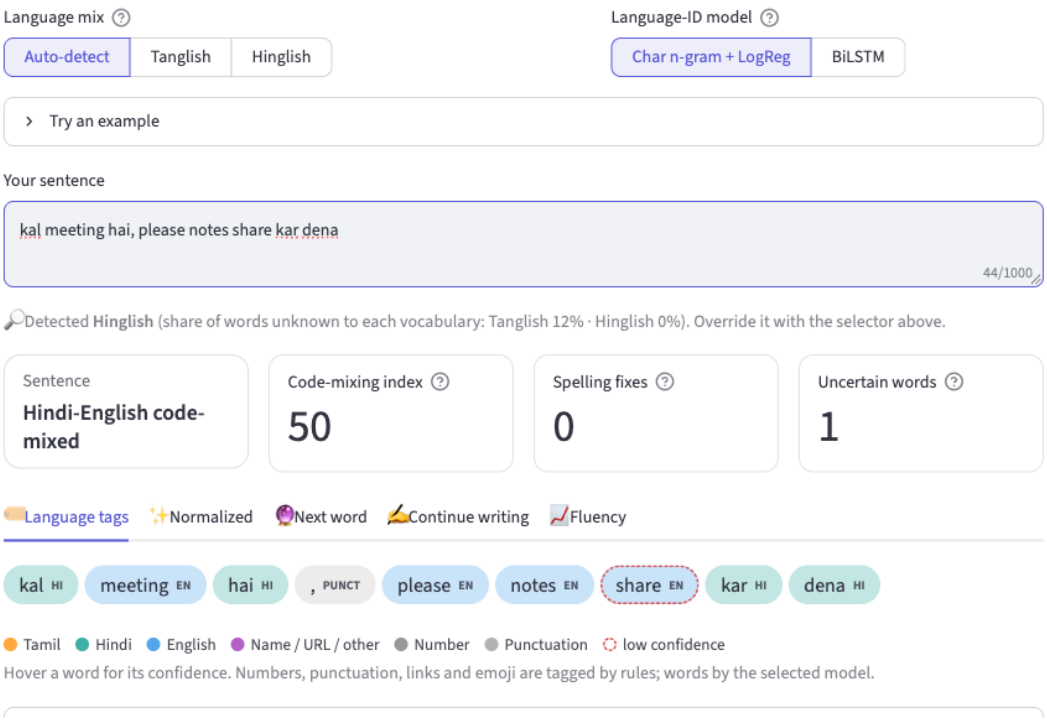


Figure 8. Hinglish input and token-level language identification (Hindi tags HI, English tags EN, punctuation by rule).

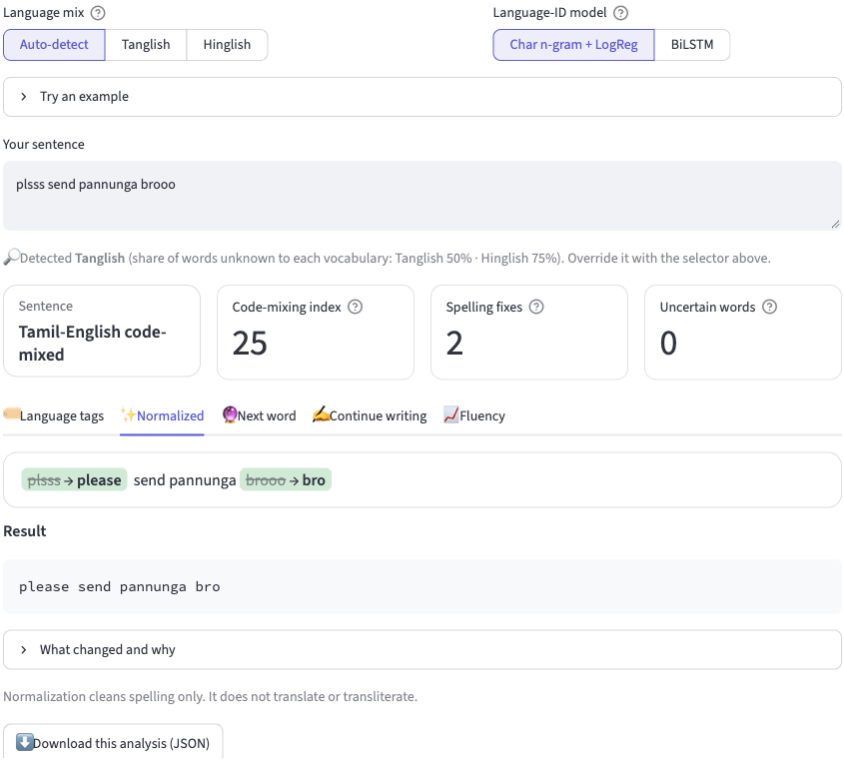


Figure 9. Text normalization of the Tanglish input plsss send pannunga brooo, giving please send pannunga bro (spelling only, no translation).

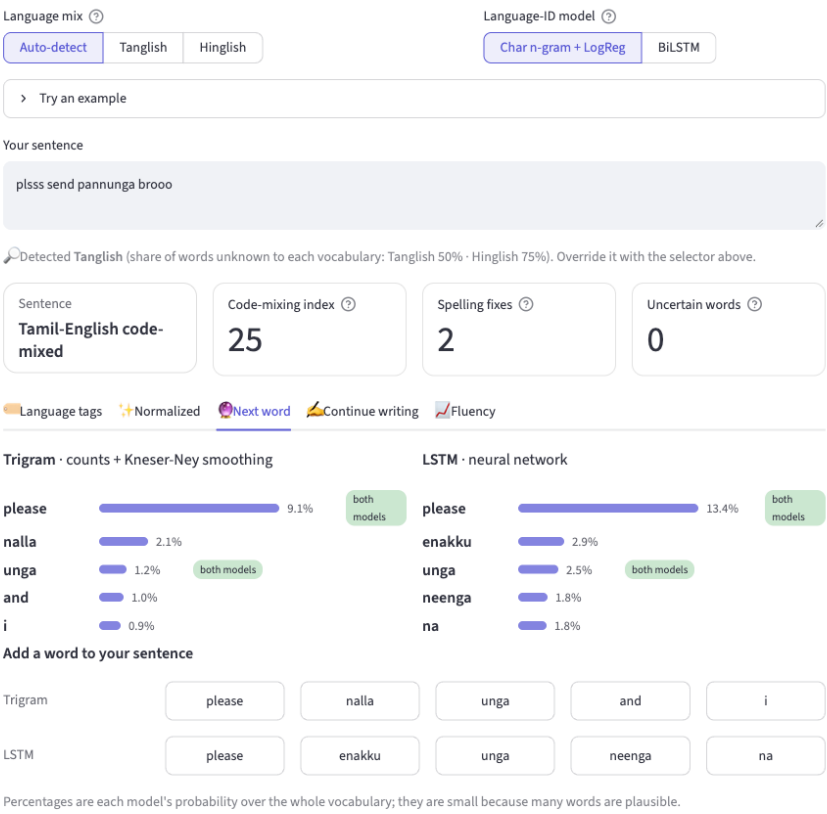


Figure 10. Next-word prediction for the Tanglish sentence: trigram (Kneser-Ney) and LSTM top-5 suggestions side by side.

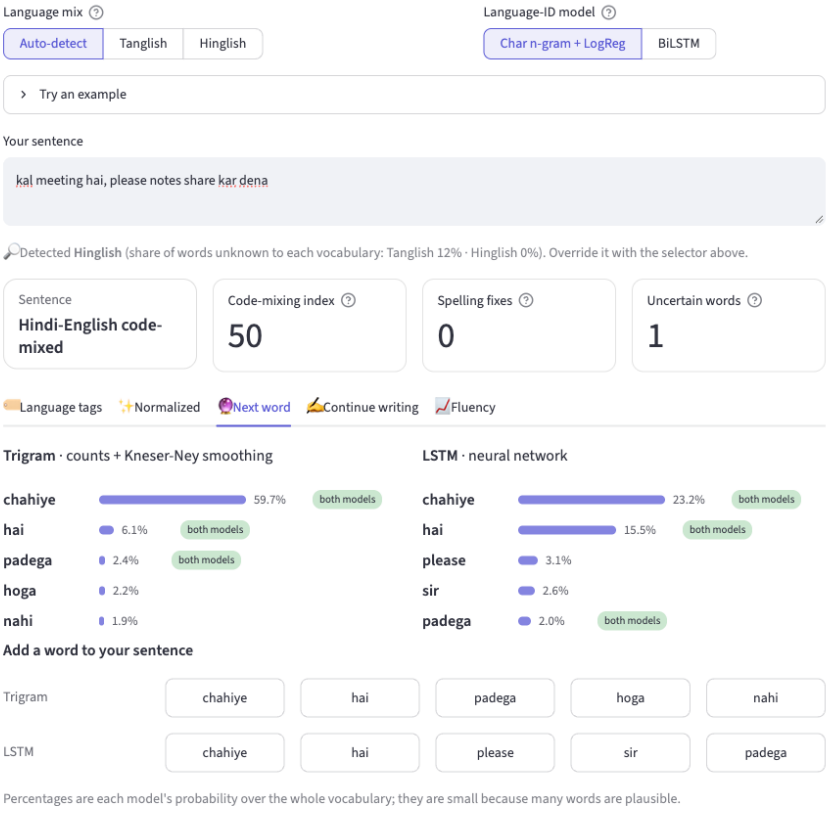


Figure 11. Next-word prediction for the Hinglish sentence: trigram (Kneser-Ney) and LSTM top-5 suggestions side by side.

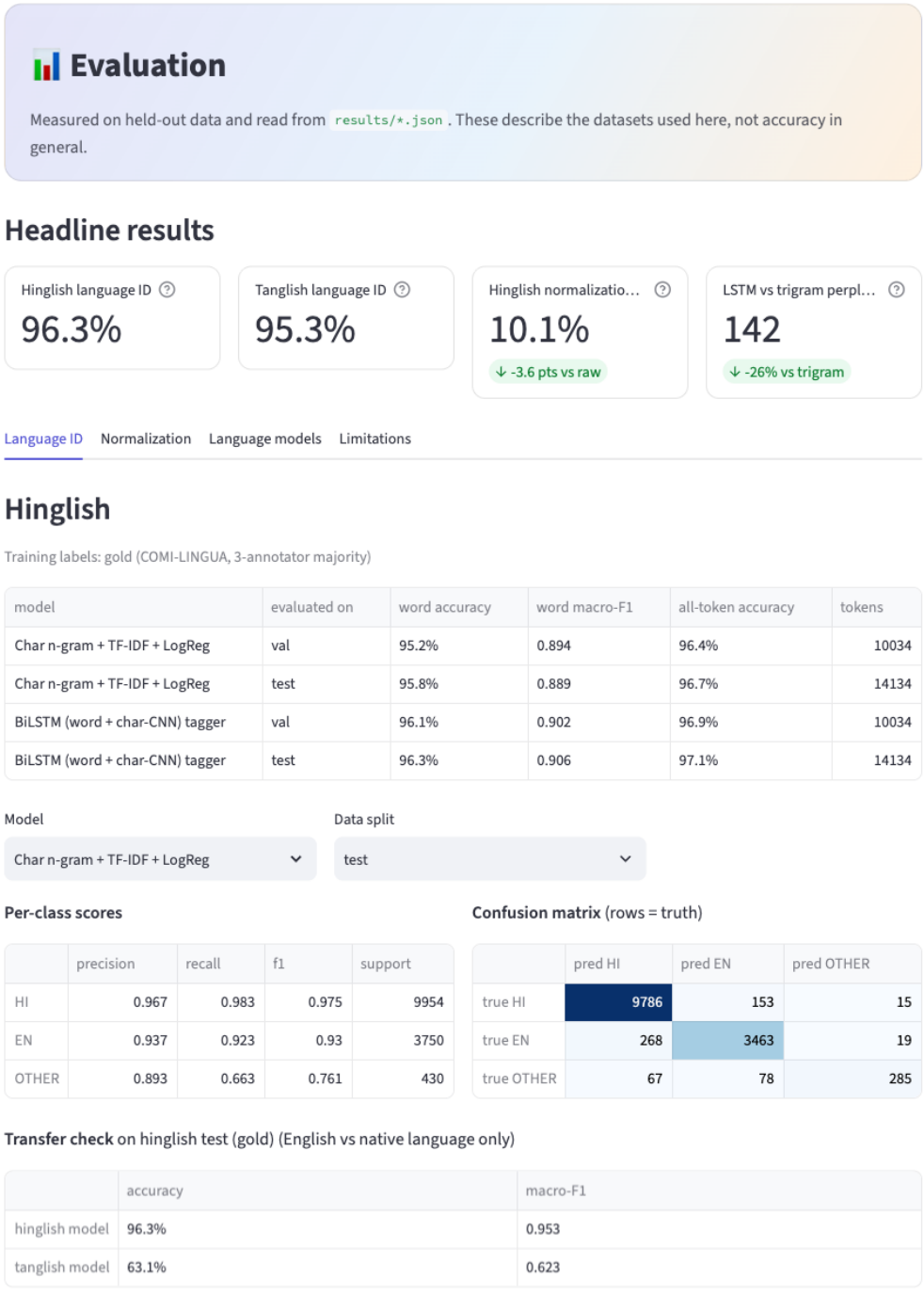


Figure 12. Evaluation dashboard: headline results (Hinglish LID 96.3%, Tanglish LID 95.3%, normalization WER 10.1%, LSTM perplexity 142), per-class scores and confusion matrix, read from results/.json (upper part of the page).*

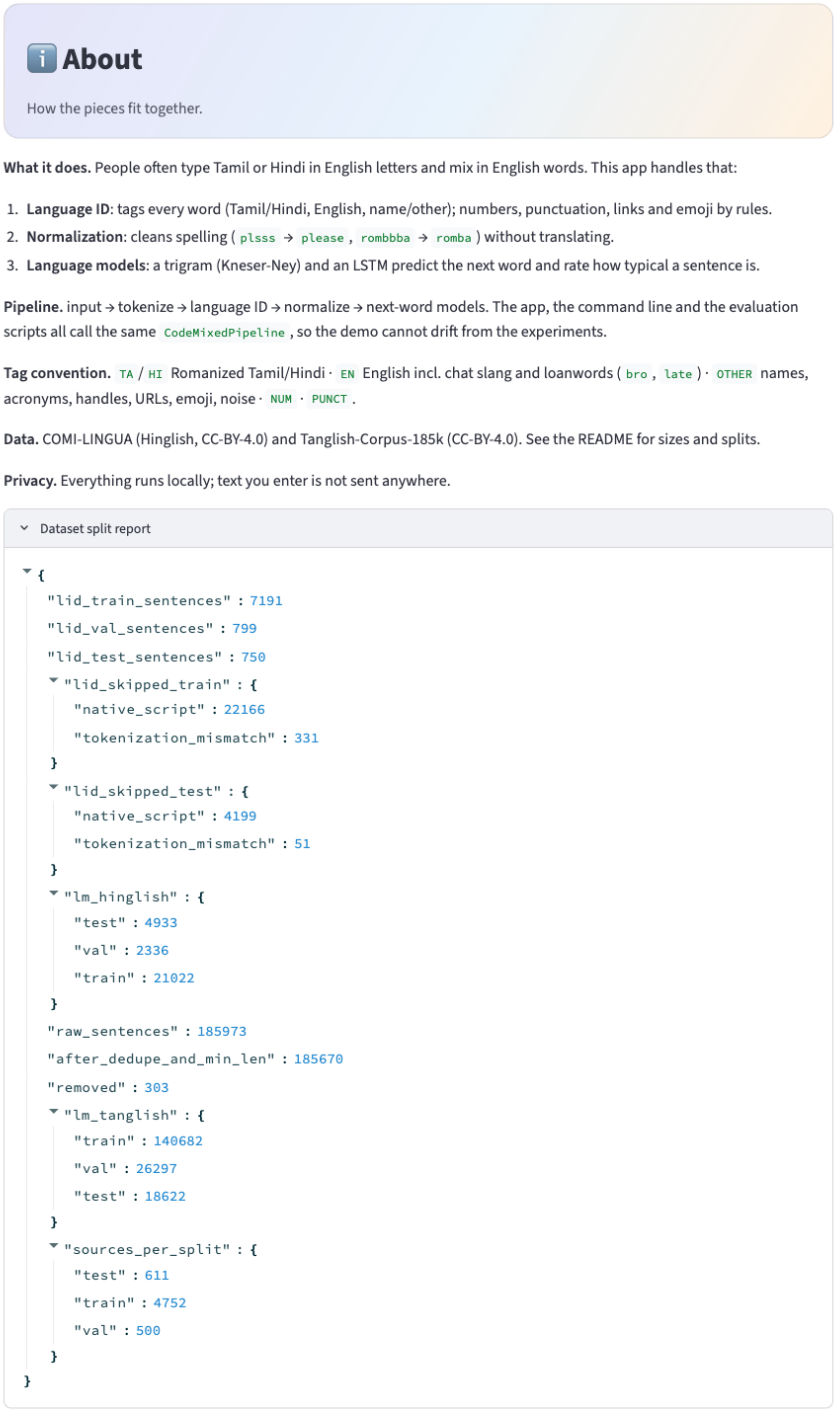


Figure 13. About page with the dataset split report (sentence counts per split), the LSTM and trigram evaluation and the data sources.

CONCLUSION

5.1 Summary of achievements

The project implemented three core NLP tasks for Romanized Tanglish and Hinglish and combined them in one tested pipeline and dashboard. Token-level language identification reached 95.8% (character n-gram) and 96.3% (BiLSTM) word accuracy on the gold Hinglish test set, and 95.3% and 95.1% on the hand-reviewed Tanglish set. Rule- and dictionary-based normalization lowered Hinglish WER from 13.6% to 10.1% against the best of three references, at a level where the human annotators themselves differ by 16.2%. Two language models were built on identical data and compared under one protocol.

5.2 Statistical versus neural models

The neural LSTM outperformed the from-scratch Kneser-Ney trigram model on every language-modelling metric: perplexity fell from 193 to 142 (Hinglish) and from 416 to 259 (Tanglish), top-1 accuracy rose by 1.2 to 1.3 points and top-5 accuracy by 2.1 to 2.2 points. The LSTM generalizes better to unseen word combinations because it shares information through embeddings, while the trigram model needs exact two-word contexts; the trigram model, however, trains in seconds to minutes, is exactly interpretable and still gives strong results. Character n-gram features alone were competitive for language identification: the BiLSTM was slightly better on Hinglish and not better on Tanglish, so sentence context added little there. These conclusions hold for these datasets and splits. Practically, code-mixed NLP of this kind supports spell-aware search, moderation and keyboard prediction for users who write in Romanized Indian languages.

5.3 Limitations

The following limitations affect how the results should be interpreted:

- **Tanglish identification labels are weak.** Training labels are generated by rules and curated dictionaries (silver labels), so the models can at best reproduce those rules; the 99% silver-test score is not generalization accuracy.
- **The Tanglish gold set is small and not independently validated.** It contains 120 sentences (1,327 word tokens) and was reviewed by the project team with an AI

assistant, not by independent native-speaker annotators; about 4% of silver labels were corrected during review.

- **Names and acronyms are weak.** Recall of the OTHER class (names, acronyms) on the Tanglish gold set is about 46%.
- **Tanglish normalization is not independently validated.** Its 97% exact match uses 68 self-authored reference pairs derived with the same dictionary conventions as the system, so it is not evidence of generalization.
- **Vocabulary cap.** The Tanglish LSTM vocabulary is capped at 20,000 words (test out-of-vocabulary rate 10.9%) because a 30,000-word vocabulary caused training problems on the MPS backend of the development machine; the cap is applied to both models.
- **No cross-variant transfer.** The Hinglish and Tanglish models do not transfer reliably to each other's language mix and must be evaluated separately.
- **Rebuild not verified.** The one-command clean rebuild `python -m src.run_all` has not been verified end to end in a single run; individual stages were run separately and the training pipeline was modified afterwards.
- **Other caveats.** Near-duplicate Hinglish tweets can exist across the official train/test split, which slightly flatters all Hinglish models; the Tanglish LSTM trained for only 10 epochs; the discount grid optimum lies at its upper edge; perplexity does not guarantee meaningful generated text; and the corpora are noisy social-media text.

5.4 Future work

- Native-speaker validation and expansion of the Tanglish gold data, with measured inter-annotator agreement.
- An independent Tanglish normalization benchmark with human references.
- Better handling of names, acronyms and other non-linguistic tokens.
- A larger vocabulary and more robust GPU training (for example sub-word or sampled-softmax output layers).
- Cross-language generalization studies, and additional Indic languages (for example Telugu, Kannada, Malayalam, Bengali).
- Language-aware normalization that uses context, and verification of the complete one-command rebuild.

REFERENCES

[1] R. Sheth, H. Beniwal and M. Singh, "COMI-LINGUA: Expert Annotated Large-Scale Dataset for Multitask NLP in Hindi-English Code-Mixing," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025 (ACL Anthology 2025.findings-emnlp.422; arXiv:2503.21670). Dataset: https://huggingface.co/datasets/LingoIITGN/COMI-LINGUA (CC-BY-4.0).

[2] U. Barman, A. Das, J. Wagner and J. Foster, "Code Mixing: A Challenge for Language Identification in the Language of Social Media," in *Proc. First Workshop on Computational Approaches to Code Switching*, Doha, Qatar, 2014, pp. 13-23. doi:10.3115/v1/W14-3902.

[3] B. Han and T. Baldwin, "Lexical Normalisation of Short Text Messages: Makn Sens a #twitter," in *Proc. 49th Annual Meeting of the ACL*, Portland, OR, 2011, pp. 368-378. https://aclanthology.org/P11-1038/

[4] A. Pratapa, G. Bhat, M. Choudhury, S. Sitaram, S. Dandapat and K. Bali, "Language Modeling for Code-Mixing: The Role of Linguistic Theory based Synthetic Data," in *Proc. 56th Annual Meeting of the ACL*, 2018, pp. 1543-1553. doi:10.18653/v1/P18-1143.

[5] B. R. Chakravarthi, V. Muralidaran, R. Priyadharshini and J. P. McCrae, "Corpus Creation for Sentiment Analysis in Code-Mixed Tamil-English Text," in *Proc. 1st Joint Workshop on Spoken Language Technologies for Under-resourced Languages (SLTU) and Collaboration and Computing for Under-Resourced Languages (CCURL)*, LREC 2020, pp. 202-210. https://aclanthology.org/2020.sltu-1.28

[6] S. F. Chen and J. Goodman, "An empirical study of smoothing techniques for language modeling," *Computer Speech & Language*, vol. 13, no. 4, pp. 359-394, 1999.

[7] R. Kneser and H. Ney, "Improved backing-off for m-gram language modeling," in *Proc. IEEE ICASSP*, vol. 1, 1995.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.

[9] Y. Kim, Y. Jernite, D. Sontag and A. M. Rush, "Character-aware neural language models," in *Proc. AAAI*, 2016. arXiv:1508.06615.

[10] O. Press and L. Wolf, "Using the output embedding to improve language models," in *Proc. 15th Conf. of the European Chapter of the ACL (Short Papers)*, Valencia, 2017, pp. 157-163.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015. arXiv:1412.6980.

[12] Vishnu N., "Tanglish-Corpus-185k," Hugging Face dataset (CC-BY-4.0). https://huggingface.co/datasets/vishnu-n/Tanglish-Corpus-185k

[13] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.